MultiProcessing

Multiprocessing - bu bir nechta mustaqil jarayonlarni parallel ravishda ishga tushirish texnologiyasi. Har bir jarayon o'z xotirasiga (memory) va resurslariga ega bo'ladi.

Threadingdan farqi
Threading: Bir jarayon ichida bir nechta threadlar (bajaruvchi ip), lekin Python GIL (Global Interpreter Lock) tufayli CPU-bound vazifalar uchun samarali emas

Multiprocessing: Har biri alohida GILga ega bo'lgan bir nechta jarayonlar, haqiqiy parallel bajarish imkonini beradi

Qo'llaniladigan joylar

Matematik hisoblar (masalan, katta matritsalar ustida amallar)

Ma'lumotlar tahlili (katta datasetlarni qayta ishlash)

Web scraping (bir vaqtning o'zida bir nechta saytlardan ma'lumot olish)

Machine learning (modelni parallel ravishda trening qilish)

Video/audio qayta ishlash

```
import multiprocessing
import time
def ishchi_funksiya(son):
    print(f"Jarayon {son} boshladi")
    time.sleep(2)
    print(f"Jarayon {son} tugadi")
if __name__ == '__main__':
    jarayonlar = []
    for i in range(5):
        p = multiprocessing.Process(target=ishchi_funksiya, args=(i,))
        jarayonlar.append(p)
        p.start()
    for p in jarayonlar:
        p.join()
    print("Barcha jarayonlar tugadi")
```

Pool - bu jarayonlar guruhi yaratish uchun mo'ljallangan vosita bo'lib, bir nechta vazifalarni avtomatik tarzda mavjud jarayonlar orasida taqsimlaydi.

Xususiyat	Oddiy `Process`	`Pool`
Yaratish	Har bir vazifa uchun alohida jarayon	Oldindan yaratilgan jarayonlar guruhi
Ishlatish	Har bir `Process` ni qo'lda boshqarish	Avtomatik tarqatish (`map`, `apply`)
Samaradorlik	Ko'p jarayonlarni qo'lda boshqarish qiyin	Optimallashtirilgan, o'zi resurslarni boshqaradi
Maqsad	Har xil turdagi vazifalar	Bir xil turdagi ko'p vazifalar

Poolda jarayonlar sonini protsessor yadrolari soniga teng qiling

```
import os
cpu_cores = os.cpu_count()  # Protsessor yadrolari soni
with Pool(cpu_cores) as p:  # Masalan, 2 yadro → Pool(2)
    p.map(...)
```

```
from multiprocessing import Pool


def hisobla(son):
    return son * son


if __name__ == '__main__':
    with Pool(processes=4) as pool:  # 4 ta jarayon guruhi yaratiladi
        natijalar = pool.map(hisobla, range(1000))  # 1000 ta sonni 4 ta
jarayonga taqsimlaydi
        print(natijalar)
```

```python
from multiprocessing import Pool
import os

def hisobla_fayl(fayl_nomi):
    with open(fayl_nomi, 'r') as f:
        tekst = f.read()
    return len(tekst.split())

if __name__ == '__main__':
    fayllar = ['file1.txt', 'file2.txt', 'file3.txt']  # 3 ta fayl bor deb faraz qilamiz
    with Pool(processes=3) as pool:  # 3 ta jarayon (har biri 1 faylni o'qiydi)
        natijalar = pool.map(hisobla_fayl, fayllar)
        print(f"Har bir fayldagi so'zlar soni: {natijalar}")
```

Asosiy tushuncha	`Multithreading`	`Multiprocessing`
Parallel ishlash	Ha, lekin **1 ta yadro**da navbatma-navbat	Ha, **bir nechta yadro**da to'g'ridan-to'g'ri parallel
Python GIL	GIL tufayli **real parallelizm emas**	GIL yo'q — **real parallelizm mavjud**
Eng mos keladigan ish turi	`I/O-bound` (kutishli) masalalar	`CPU-bound` (hisobli, og'ir) masalalar
Xotira	Kamroq sarf bo'ladi	Har bir process alohida xotira → ko'proq RAM
Ma'lumot ulashish	Oson (`shared memory`)	Qiyinroq (`Queue` , `Pipe`) kerak bo'ladi
Ishga tushirish	Tez, engil	Sekinroq, og'irroq
Qayerda ishlatiladi	Web scraper, GUI, fayl o'qish, tarmoqlar	ML training, video processing, og'ir matematik hisoblar

Afzalliklari

Haqiqiy parallel bajarish: Har bir jarayon alohida CPU yadrosida ishlashi mumkin

GIL cheklovidan qochish: Har bir jarayon o'z GILiga ega

Xotira izolyatsiyasi: Jarayonlar bir-birining xotirasiga ta'sir qilmaydi

Barqarorlik: Bir jarayondagi xato boshqalariga ta'sir qilmaydi

Kamchiliklari

Jarayonlar o'rtasida ma'lumot almashish qiyin: Threadingga qaraganda ancha sekin

Yangi jarayon yaratish vaqt oladi

Xotira iste'moli ko'p: Har bir jarayon o'z nusxasini yaratadi